ESP32 WebSockets Guide

(Absolute Beginner Step-by-Step Tutorial)

Overview

This guide walks you through setting up a real-time, two-way WebSocket communication link between an ESP32 board and a web browser. Unlike traditional HTTP requests where the client must constantly ask for updates (polling), WebSockets keep a connection open, allowing the ESP32 to instantly push data to a webpage the moment it happens.

Prerequisites & Hardware Needed

- 1x ESP32 Microcontroller Board (e.g., ESP32 Dev Module / NodeMCU)
- Micro-USB or USB-C Data Cable (Ensure it supports data transfer, not power-only)
- Windows/Mac PC with Arduino IDE 2.x installed
- A local Wi-Fi network (Router)

Step 1: Install Required Libraries in Arduino IDE

1. Open Arduino IDE and go to **Tools > Manage Libraries...** (or press Ctrl + Shift + I).
2. In the search box, type WebSockets.
3. Locate **WebSockets** by *Markus Sattler* and click **Install**.
4. If prompted to install missing dependencies, click **Install All**.

Step 2: ESP32 WebSocket Server Code

Create a new sketch in Arduino IDE (**File > New Sketch**). Replace all existing code with the following snippet.

Important: Make sure to replace YOUR_WIFI_SSID and YOUR_WIFI_PASSWORD with your actual Wi-Fi credentials.

[Use the file of Websocket.ino]

Step 3: Upload Code and Get the IP Address

1. Plug in your ESP32 board via USB.
2. Select your board under **Tools > Board > esp32 > ESP32 Dev Module**.
3. Select the active COM port under **Tools > Port**.
4. Click the **Upload** button.
5. Once uploaded, open the **Serial Monitor** (Ctrl + Shift + M).
6. Set the baud rate dropdown to **115200 baud**.
7. Press the **EN** (or Reset) button on your ESP32.

8. Watch the Serial Monitor. Once connected to Wi-Fi, it will print the **ESP32 IP Address** (e.g., 192.168.1.50). Write this down.

Step 4: Create the Client Webpage

You don't need a web server for this step; you can run the HTML file directly from your computer.

1. Open a simple text editor (like Notepad on Windows or TextEdit on Mac).
2. Paste the following HTML/JavaScript code.
3. **Crucial:** Change the wsUri variable to match the IP address you copied in Step 3.
4. Save the file as websocket_client.html.

[Use the file ofWebPage.txt]

Step 5: Verify Communication

1. Double-click the websocket_client.html file you just saved to open it in your web browser.
2. Click the **Connect** button on the webpage.
3. You should see "CONNECTED" appear, followed quickly by the "Welcome" message from the ESP32.
4. Every 5 seconds, you will see a "Server update" message pushed from the ESP32 to the webpage.
5. Type a message in the input box and click **Send to ESP32**. You should see it sent, and then immediately receive the "ESP32 Echo:" response back.
6. Check the Arduino Serial Monitor; you will see the connection events and the text you sent from the browser printed there.

Common Mistakes & Troubleshooting

1. Using delay() in loop():

- **Mistake:** Adding long delay(1000) calls inside the main loop().
- **Why it breaks:** webSocket.loop() must be called constantly to process incoming and outgoing network traffic. If delay() pauses the ESP32, the WebSocket connection will time out, drop packets, or disconnect the client entirely.
- **Fix:** Use non-blocking timers with millis() (as demonstrated in the example code) to schedule tasks without freezing the processor.

2. Forgetting to Call webSocket.loop():

- **Mistake:** Putting code in the loop() but omitting webSocket.loop();.
- **Why it breaks:** The server initializes, but it never actively listens for new client connections or processes data from existing ones. The system appears completely dead.
- **Fix:** Ensure webSocket.loop(); is executed as frequently as possible inside the main loop().

3. Mixing Up WS and HTTP Ports:

- **Mistake:** Trying to connect to ws://192.168.1.50:80/ instead of port 81.
- **Why it breaks:** WebSockets and standard HTTP web servers often run simultaneously on an ESP32. Usually, HTTP runs on port 80 and WebSockets on port 81. If you point your JavaScript to the wrong port, the connection will be refused.
- **Fix:** Double-check that your WebSocketsServer initialization (WebSocketsServer(81)) matches the port in your JavaScript URI (ws://YOUR_IP:81/).

4. Sending Huge Payloads:

- **Mistake:** Trying to send massive strings or huge JSON objects in a single sendTXT or broadcastTXT command.
- **Why it breaks:** The ESP32 has limited RAM. Constructing huge strings can cause memory fragmentation or stack overflows, leading to a crash and reboot (Watchdog Timer reset).
- **Fix:** Break large amounts of data into smaller chunks before sending, or increase the WebSocket buffer size in the library settings (advanced).

5. Ignoring Wi-Fi Dropouts:

- **Mistake:** Assuming the Wi-Fi connection will never fail after the initial setup.
- **Why it breaks:** If the router reboots or the signal drops, the WiFi.status() becomes disconnected, and the WebSocket server ceases to function.
- **Fix:** Implement a Wi-Fi auto-reconnect routine in the loop() to check the connection status and attempt to reconnect if it drops.